
Supplementary Material for Beacon: Agent-Readable Function Registries for the Long Tail of Scientific Python

Anonymous Author(s)

Affiliation

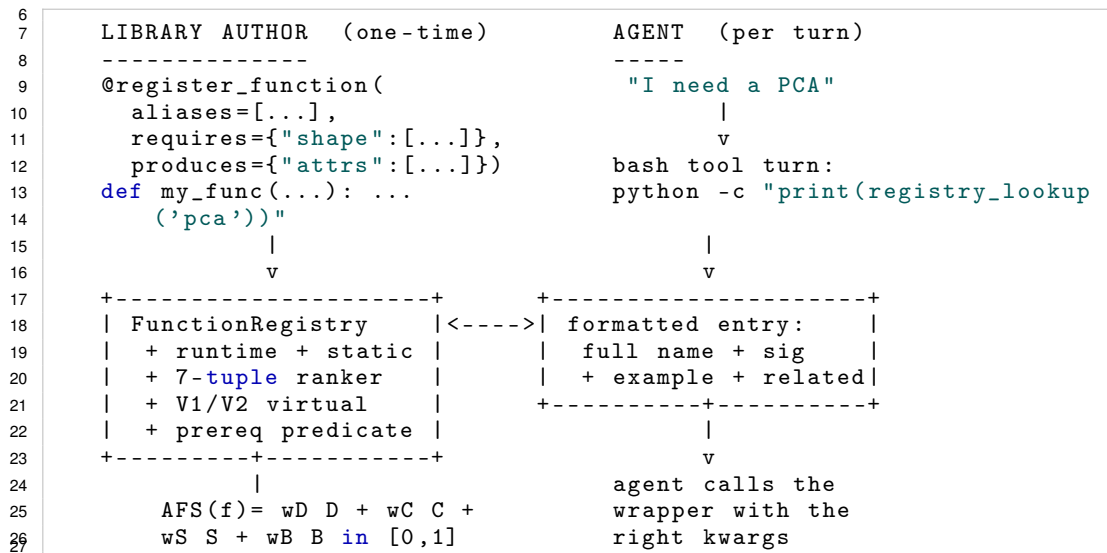
Address

email

1 Supplementary Material

2 This supplement accompanies the paper *Beacon: Agent-Readable Function Registries for the Long*
3 *Tail of Scientific Python*. Section numbers below are prefixed with S to disambiguate from main-text
4 references.

5 1 §S0 System architecture



28 The framework has three artefacts the library author touches and two interfaces the agent
29 calls. Authors decorate at the call site (§3.2) and audit by AFS (§3.7); the run-
30 time registry, the static AST scanner (the fallback path for sandboxes that cannot im-
31 port the package), and the skill registry compose into the two lookup APIs (§3.4) the
32 agent reaches via a single bash turn. Both standalone artefacts are released: *awe-*
33 *agent-function* (<https://anonymous.4open.science/r/awe-agent-function-E720/>) ships
34 the decorator, registry, scanner, and AFS audit; the companion *awesome-skill-generate*
35 (<https://anonymous.4open.science/r/awesome-skill-generate-17E3/>) ships the 12-
36 principle content contract and the 9-dimension rubric for the Markdown skill bodies.

37 The figure shows the three author-side artefacts (decorator, registry contract, AFS audit) and the two
 38 agent-side interfaces (registry_lookup, skill_lookup) that compose at plan time.

39 2 §S1 The decorator entry schema

40 A @register_function invocation, after validation, stores a canonical entry of the following shape
 41 in the process-global FunctionRegistry:

```

42 {
43   "function":          <Callable>,          # the wrapped object
44   "full_name":         "omicverse.pp._qc.qc",
45   "short_name":        "qc",
46   "module":            "omicverse.pp._qc",
47   "aliases":           [...],                # the decorator's
48   aliases= list
49   "category":          "preprocessing",
50   "description":        "Perform comprehensive quality control ...",
51   "examples":          [...],                # one-line ov.*
52   examples
53   "related":           [...],                # cross-references
54   "signature":          "(adata, mode='seurat', tresh=None, ...)",
55   "parameters":         ["adata", "mode='seurat'", ...],
56   "docstring":          inspect.getdoc(func),
57   "prerequisites":      {"functions": [...], "optional_functions":
58   [...]}},
59   "requires":           {"layers": [...], "obs": [...], ...},
60   "produces":           {"layers": [...], "obs": [...], ...},
61   "auto_fix":           "auto" | "escalate" | "none",
62   "virtual_entry":      False,                # True for V1 / V2
63   "source":             "runtime",            # vs "static_ast" for
64   scanner
65 }
66

```

68 The eight decorator arguments are validated as follows. (i) aliases must be a non-empty list of
 69 non-empty strings. (ii) category and description must be non-empty. (iii) prerequisites
 70 accepts only the keys {functions, optional_functions}. (iv) requires and produces each
 71 accept only the seven AnnData-container keys {layers, obs, obsp, uns, var, obs, varm}.
 72 (v) auto_fix must be one of {auto, escalate, none}. (vi) Any malformed input raises at
 73 decoration time, which means at module-import time — broken metadata is impossible to ship.

74 The decorator then inspects the function with inspect.signature to recover the signature string
 75 with all keyword-only arguments and their defaults. Finally it fans the entry out across multiple keys
 76 in the registry dict (lower-cased short name, every lower-cased alias, full dotted name) so that any of
 77 those queries returns the same record in $O(1)$.

78 3 §S2 Lookup scoring algorithm in full

79 For query q and entry e the lookup ranks the merged registry by a 7-tuple lexicographic sort key
 80 $\sigma(q, e) = (r_1, s_2, s_3, k_4, s_5, s_6, r_7)$.

81 Let $T(q)$ denote the lower-cased query tokens — extracted by the regex
 82 $[a-z0-9_.\-]+\mid[U+4E00-U+9FBF]+$ (ASCII alphanumerics plus the CJK Unified Ideographs
 83 block) so that mixed English / Chinese queries tokenise cleanly. For a field f define the **token**
 84 **overlap score**

$$\text{TO}(q, f) = \sum_{t \in T(q)} (2 \cdot \mathbb{K}[t = \tilde{f}] + \mathbb{K}[t \neq \tilde{f} \wedge t \subset \tilde{f}]),$$

85 where $\tilde{f} = \text{lower}(f)$ and $\subset$ is the substring relation. The seven components are then:

86 **(1) Exact rank.**

$$r_1(q, e) = \begin{cases} 4 & q \in \{\text{full_name}(e), \text{registry_full_name}(e)\} \\ 3 & q = \text{short_name}(e) \vee q \in A(e) \\ 0 & \text{otherwise.} \end{cases}$$

87 **(2) Alias score.** With strong-field set $F_{\text{str}}(e) = \{\text{full_name}, \text{registry_full_name}, \text{short_name}\}(e)$,

$$s_2(q, e) = 2 \cdot \mathbb{K}[\exists f \in F_{\text{str}}(e) : q \subset f] + \max_{a \in A(e)} \text{TO}(q, a) + \mathbb{K}[\exists a \in A(e) : q \subset a].$$

88 **(3) Name score.**

$$s_3(q, e) = \max_{f \in F_{\text{str}}(e)} (\text{TO}(q, f) + \mathbb{K}[q \subset f]) + \mathbb{K}[\exists t \in T(q) : t = \text{short_name}(e)].$$

89 **(4) Kind score** prefers parents over methods over branches:

$$k_4(e) = \begin{cases} 0 & e \text{ is a virtual branch entry (V2)} \\ 1 & e \text{ is a virtual method entry (V1)} \\ 2 & e \text{ is a registered top-level entry.} \end{cases}$$

90 **(5) Weak score.** With weak-field set $F_{\text{wk}}(e) = \{\text{category}, \text{description}, \text{docstring}, \text{examples}, \text{related}, \text{imports}\}(e)$
 91 and domain bias sets $\mathcal{D}_{\text{ds}} = \{\text{dataset}, \text{download}, \text{read}, \text{load}, \text{example}, \text{demo}\}$, $\mathcal{D}_{\text{cr}} =$
 92 $\{\text{reference}, \text{table}, \text{gpu}, \text{cpu}, \text{settings}\}$,

$$s_5(q, e) = \sum_{f \in F_{\text{wk}}(e)} \text{TO}(q, f) \\ - \mathbb{K}[\text{full_name}(e) \text{ starts with } \text{ov.datasets.}] \cdot \mathbb{K}[T(q) \cap \mathcal{D}_{\text{ds}} = \emptyset] \\ - \mathbb{K}[\text{full_name}(e) \text{ starts with } \text{ov.core.}] \cdot \mathbb{K}[T(q) \cap \mathcal{D}_{\text{cr}} = \emptyset].$$

93 **(6) Contract score.**

$$s_6(q, e) = \sum_{f \in \{\text{flat}(\text{prereq}), \text{flat}(\text{req}), \text{flat}(\text{prod})\}} (\min(\text{TO}(q, f), 1.5) + 0.25 \cdot \mathbb{K}[q \subset \tilde{f}]).$$

94 **(7) Source rank** $r_7(e) = \mathbb{K}[\text{source}(e) = \text{runtime}]$ — a small runtime-vs-static tiebreak.

95 The merged registry is sorted by σ in lexicographic descending order; entries with all zeros on
 96 $\{r_1, s_2, s_3, s_5, s_6\}$ are dropped; the top $k = 8$ are returned. A scalar projection $\hat{\sigma} = 10^3 r_1 +$
 97 $10^2 s_2 + 10 s_3 + k_4 + 10^{-1} s_5 + 10^{-2} s_6 + 10^{-3} r_7$ is also exposed for trace logging; it preserves the
 98 lexicographic order under the bounds of the bench's check counts. The full implementation is roughly
 99 110 lines.

100 3.1 §S2.1 Sample formatted output

101 A real registry_lookup("pca") returns:

```
102 [1] omicverse.pp.pca.pca
103 desc:      Principal component analysis on a feature matrix...
104 aliases:   pca, PCA, principal component analysis
105 category:  preprocessing
106 signature: (adata, n_pcs=50, layer='scaled', use_highly_variable=
107             True, ...)
108 examples:
109     ov.pp.pca(adata, n_pcs=50)
110     ov.pp.pca(adata, n_pcs=30, layer='scaled')
111 related:   neighbors, umap, leiden
112 requires:  layers:scaled
113 produces:  obsm:X_pca, varm:PCs, uns:pca
```

The signature line is the AST-derived form including keyword-only arguments and their defaults, so dispatch knobs like `doublets_method='scrublet'` are visible to the agent on first read. In an earlier version of the package the signature was extracted only from `inspect.signature`'s positional view, which omitted dispatch parameters and forced the agent to re-discover them through trial-and-error `AttributeErrors`.

§S2.4 AFS weight sensitivity

The default AFS weights $(w_D, w_C, w_S, w_B) = (0.30, 0.35, 0.15, 0.20)$ are chosen by the cost-of-failure argument of §3.7. We test whether the *ranking* AFS induces over a library's functions is robust to the specific weights. For each library we compute the default-weight ranking π^* , then sample $N = 500$ perturbed weight vectors $w + \delta$ with each component $\delta_i \sim \text{Uniform}[-0.10, +0.10]$, renormalised to sum to 1 and clipped at 0.05 below. For each perturbation we compute the perturbed ranking π and report Spearman $\rho(\pi^*, \pi)$.

Library	Mode	N	$\bar{\rho}$	$\rho_{\min}$	$\bar{\rho}$ at ± 0.20
omicverse	hand	862	0.986	0.916	0.964
sklearn	auto	301	0.953	0.849	0.918

Even under doubled perturbation envelopes (± 0.20), the rank correlation to the default-weight ranking remains above 0.91 on both libraries. Extreme single-component-dominant weight choices (one weight = 0.70, others = 0.10) preserve $\rho \in [0.77, 0.97]$. We conclude that the AFS *ordering* — which is what a maintainer uses when prioritising the next field to populate — is not a function of the specific weight choice in §3.7. Predictive validity across libraries (whether $\Delta\text{Pass@1}$ scales with ΔAFS across libraries) is a separate question that requires multi-library agent results we have not yet collected (§8 L4).

§S2.5 Six-library AFS audit

We run the §3.7 audit over six libraries to test whether AFS (a) discriminates the hand vs. auto regimes Beacon supports and (b) varies informatively across mainstream domains. The auto-decoration is the same runtime walker exercised on sklearn in §7.2, applied recursively to depth 2 over each library's public surface.

Library	Mode	N	AFS	D	C	S	B
omicverse	hand	862	0.65	0.99	0.33	0.24	1.00
sklearn	auto	301	0.53	0.91	0.00	0.68	0.79
networkx	auto	967	0.50	0.94	0.00	0.83	0.46
scipy	auto	1157	0.46	0.88	0.00	0.53	0.61
numpy	auto	567	0.43	0.88	0.00	0.28	0.63
pandas	auto	352	0.43	0.94	0.00	0.37	0.44

On the $C = 0$ column. Every auto-decorated mainstream library has $C = 0.00$ by construction: docstrings do not encode `requires / produces / auto_fix`, and the auto-decorator does not hallucinate them. We therefore do not report this as a discovery. The interpretive content is what *the other three* sub-scores look like under docstring-only synthesis, and how they vary across libraries. (a) Mainstream auto-decorated libraries cluster at $\text{AFS} \in [0.43, 0.53]$ regardless of size or popularity (sklearn 301 entries, scipy 1157, pandas 352). Library size does not move the score; the docstring-style ceiling does. (b) Sub-score variation reflects authoring-style choices: `networkx`'s consistently keyword-argument-shaped public API drives $S = 0.83$, while `pandas`'s prose-heavy docstring style with few `Examples` blocks drives $B = 0.44$. The audit gives a per-library punch-list — for `pandas` "populate `Examples` sections," for `numpy` "expose keyword-argument defaults," for any auto-decorated library "author `requires / produces`" — without a single agent trajectory.

153 4 §S3 The skill content contract: 12 principles + 9-dimension rubric

154 A skill is free-text Markdown and admits much more variation in quality than a typed function. We
155 propose a 12-principle content contract that a skill body must satisfy and a 9-dimension audit rubric
156 for measuring it. Reference implementation in *awesome-skill-generate*.

157 4.1 §S3.1 The twelve principles

158 1. **Capability-first naming.** The frontmatter is the search
159 target; phrase the skill in terms of the analytical decision the agent is making, not the source notebook.

160 1. **Single-job boundaries.** One skill answers one workflow
161 question. Skills that mix two workflows split unless the shared boundary is itself the contract being
162 taught.

163 1. **Trigger / execution / validation contracts.** Every skill
164 body has three explicit zones: when to invoke, the sequence of registered functions to call, how
165 the harness or reviewer recognises success. The validation zone composes with §3.5's prerequisite
166 predicate.

167 1. **Source-grounded APIs.** Every concrete API call in a skill
168 body resolves against the live registry — the function exists, the signature matches, the dispatch
169 string is in the documented set. A separate validator runs `inspect.signature` against every quoted
170 call.

171 1. **Branch coverage.** A skill that names a `method=` parameter
172 enumerates the working values, in correspondence with §3.3's V2 virtual entries.

173 1. **Environment-agnostic.** Skill bodies use repo-relative paths
174 and import paths, not absolute filesystem paths or machine-specific conda env names.

175 1. **Smoke-path validation.** Long-running workflows ship a
176 bounded smoke test the agent may execute first.

177 1. **Empirical executability.** Data-heavy workflows include
178 reviewer-side execution evidence (an example AnnData snapshot, expected check pass / fail).

179 1. **Structured artefacts.** A skill is a directory:
180 `<slug>/SKILL.md` (frontmatter + trigger + execution + validation), `references/`, `assets/`, op-
181 tional `scripts/`.

182 1. **Human-readable scoring.** Every skill is reviewed against
183 the rubric below and produces a per-dimension score with a written justification.

184 1. **Rubric-based measurement.** Quality is measured against
185 explicit criteria, not intuition.

186 1. **Explicit skill contracts.** A skill is not a tutorial; the
187 trigger / execution / validation blocks are explicit.

188 4.2 §S3.2 The nine-dimension quality rubric

189 Each dimension scored on a 1–5 scale; default uniform weights.

	Dimension	What it measures
	Trigger Precision	Accuracy of invocation conditions
	Execution Clarity	Understandability of the workflow steps
	Validation Strength	Robustness of acceptance checks
190	Empirical Executability	Ability to verify via live execution
	Context Efficiency	Appropriate scope and dependencies
	Reusability	Portability across contexts
	Resource Partitioning	Clean separation of concerns
	Compatibility Robustness	Resilience to API / environment changes
	Maintainability	Updateability as upstream code evolves

191 The aggregate is $Q(s) = \sum_d w_d q_d(s)$ with $q_d(s) \in \{1, 2, 3, 4, 5\}$ and a default uniform weighting, so
192 $Q_{\max} = 45$ (or $5 \times 20 = 100$ when re-scaled, the convention used by the reference implementation).
193 Inclusion threshold: $Q \geq 0.85 \cdot Q_{\max}$.

194 The exemplar dynamo-preprocess skill released alongside the paper sits at $Q = 0.95 \cdot Q_{\max}$.

195 5 §S4 Reproducibility

196 5.1 §S4.1 Released artefacts

197 • awe-agent-function:

198 <https://anonymous.4open.science/r/awe-agent-function-E720/> — the decorator, reg-
199 istry, scanner, lookup, AFS audit, plus four runnable examples and a tutorial notebook.

200 • awesome-skill-generate:

201 <https://anonymous.4open.science/r/awesome-skill-generate-17E3/> — the 12-
202 principle content contract, the 9-dimension rubric, and the exemplar skill.

203 • omicverse: 445 @register_function decorations across

204 158 modules.

205 • ovagent-bench: the runner, the grader, the failure-mode

206 tagger, the report builder, the YAML sweep configurations, and the SHA-256-pinned fixtures.

207 5.2 §S4.2 Repository layout (released code)

```

208 ovagent-bench/
209 |-- bench/                                # python package
210 |   |-- tasks.py                          # 24 v1.0 tasks + 16 extension
211 |   tasks
212 |   |-- runner.py                         # YAML-driven trajectory generator
213 |   |-- grader.py                         # 65 atomic check types
214 |   |-- grade_run.py                     # trajectory JSON -> grades.csv
215 |   |-- failure_taxonomy.py              # 7-class tagger
216 |   |-- stats.py                         # bootstrap CI + paired McNemar +
217 |   BH-FDR
218 |   |-- adapters/
219 |   |   |-- mini_swe.py                  # shared loop for both arms
220 |   |   |-- human_scanpy.py             # hand-coded reference baseline
221 |-- prompts/
222 |   |-- omicverse_system.md              # the 222-line system prompt
223 |-- tasks_md/                            # per-task spec files (24 +
224 |   extensions)

```

```

226 | -- configs/                                # one YAML per reproducible sweep
227 | -- fixtures/
228 | | -- manifest.json                        # SHA-256 + size of each fixture
229 | | -- inputs/                             # raw input data
230 | | -- oracles/                             # gold final-state for ARI /
231 | Jaccard
232 | -- results/                               # per-run JSON + grades.csv

```

234 §S4.3 Statistical methodology

235 For model m , arm $a \in \{B, O\}$, task set $\mathcal{T}_m$, seed s , and per-trajectory pass indicator $y_{m,a,\tau,s} \in \{0, 1\}$,
 236 the headline statistic is

$$\text{Pass@1}_{m,a} = \frac{1}{|\mathcal{T}_m|} \sum_{\tau \in \mathcal{T}_m} \frac{1}{|\mathcal{S}_m|} \sum_{s \in \mathcal{S}_m} y_{m,a,\tau,s}$$

237 (per-task averaged across seeds first, then across tasks). For six of seven models $|\mathcal{S}_m| = 3$ (seeds 0, 1,
 238 2); for qwen-3B-a3b $|\mathcal{S}_m| = 1$ (seed 0 only — locally served via ollama, sweep cost is multi-day).
 239 The within-model uplift is $\Delta_m = \text{Pass@1}_{m,O} - \text{Pass@1}_{m,B}$; the panel-level summary is the simple
 240 mean $\bar{\Delta} = |\mathcal{M}|^{-1} \sum_m \Delta_m$.

241 Pass@1 confidence intervals use 10,000-sample percentile bootstrap over the per-task pass indicators.
 242 Per-cell paired McNemar at (τ, s) granularity is the default significance test; Benjamini–Hochberg
 243 FDR at $q = 0.10$ is applied across system-pair contrasts. The strongest distribution-free statement
 244 we make about Table 2 is the panel-wide paired sign test: under $H_0 : \Pr(\Delta_m > 0) = 1/2$ with
 245 $|\mathcal{M}| = 7$, the one-sided p -value for $K = 7$ positive contrasts is $p = \binom{7}{1} 2^{-7} = 1/128 \approx 0.008$.

246 §S4.4 Hardware and serving

247 qwen-3B-a3b-256k is served via local ollama on a single A100 / H100 node. gpt-5.5,
 248 gemini-3.1-flash-lite-preview, deepseek-v4-flash, deepseek-v4-pro, glm-5.1, and
 249 MiniMax-M2.7 are served via API. Both arms share a single conda environment (omicdev) on the
 250 worker that executes bash subshells; this environment ships omicverse, the scverse stack (anndata,
 251 scanpy, scvelo, mudata, muon), and the Python interpreter the agent invokes.

252 The agent loop is mini-swe-agent with a single bash tool per turn, hard caps at max_turns
 253 (default 30) and wallclock_s_limit (default 600–1200 s depending on layer). Per (task, seed,
 254 system, model) the runner emits one trajectory JSON aligned with the BixBench schema. The grader
 255 is a post-processing pass that consumes the trajectory and the final AnnData and emits per-check pass
 256 / fail records, so trajectories can be re-graded under future check-type changes without re-running the
 257 agent.

258 §S4.5 Per-trajectory efficiency

259 §S4.6 Ablation figure (companion to Table 3)

260 §S5 Prerequisite-checking API

261 The decorator’s requires map defines a runtime predicate over an AnnData **A**. Let $\mathcal{C} =$
 262 $\{\text{layers, obsm, obsp, uns, var, obs, varm}\}$ be the seven containers. With $R(f) \subseteq \mathcal{C} \times \Sigma^*$ the set
 263 of required (container, key) pairs,

$$\text{satisfied}(f, \mathbf{A}) = \bigwedge_{(c,k) \in R(f)} \mathbb{1}[k \in \mathbf{A}.c].$$

264 The runtime checker invocation:

```

265 result = registry.check_prerequisites("pca", adata)
266

```

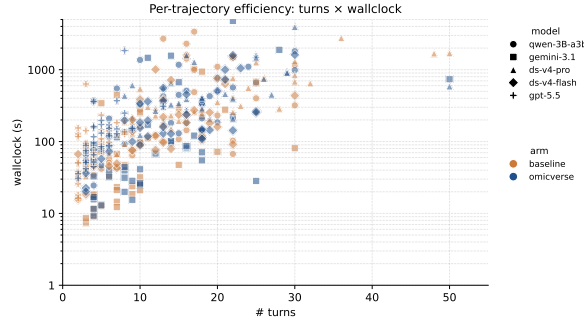


Figure 1: **Per-trajectory efficiency.** Turns (x) vs. wallclock seconds (y, log scale), one point per trajectory across the v1.1 38-task panel $\times$ 5 models $\times$ 2 arms ($\approx$ 380 trajectories). Marker shape encodes model, color encodes arm. gpt-5.5 clusters in the lower-left (2–13 turns, 10–1000 s); deepseek-v4-pro carries the right tail (up to 50 turns, 1000–5000 s); qwen3.6:35b-a3b stretches up the wallclock axis because of local ollama serving. The within-model arm separation is small for every model except gemini-3.1, where the +Beacon arm sits slightly to the right (extra registry_lookup / skill_lookup calls cost a turn each).

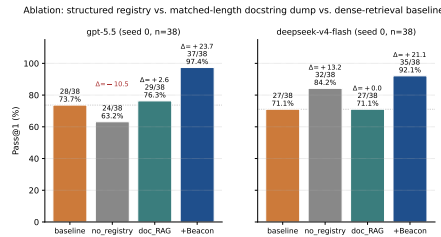


Figure 2: **Four-arm ablation on gpt-5.5 and deepseek-v4-flash** (seed 0, $n = 38$; numbers in §6.5 / Table 3). *baseline*: no registry, no discipline prompt. *no_registry*: same 222-line discipline prompt + matched-length flat docstring dump (structured fields stripped). *doc_RAG*: sentence-embedding retriever (all-MiniLM-L6-v2, top-8) over raw docstrings, injected into the same prompt frame. *+Beacon*: full structured registry. Dotted line = baseline. On gpt-5.5 the no-registry arm underperforms baseline by 10.5 pp; doc-RAG buys +2.6; +Beacon +23.7. On deepseek-v4-flash discipline alone helps somewhat (+13.2), doc-RAG returns to baseline (0), +Beacon still leads by +21.1.

```

267 # {
268 #   "satisfied":      False,
269 #   "missing_functions": ["scale"],
270 #   "missing_structures": ['adata.layers["scaled"]'],
271 #   "recommendation": "Run ov.pp.scale() first or use ov.pp.
272 #                     preprocess()",
273 #   "auto_fixable":   False,
274 # }

```

276 The companion helper renders the prerequisite chain as a five-line prompt block ready to drop into a
 277 system prompt:

```

278 Function: omicverse.pp.pca()
279 Prerequisites:
280 - Required functions: scale
281 - Optional functions: qc, preprocess
282 - Requires: adata.layers['scaled']
283 - Produces: adata.obsm['X_pca'], adata.varm['PCs'], adata.uns['pca']
284 Prerequisite Chain: scale -> pca
285 Full Chain (with optional): qc -> preprocess -> scale -> pca
286 Auto-fix Strategy: ESCALATE (suggest workflow for complex cases)
287
288

```


289 The `auto_fix` strategy decides what to do with the missing set: `auto` allows an agent or harness
 290 to silently insert up to two prerequisite calls; `escalate` recommends a workflow function; `none`
 291 only warns. The two-call cap on auto-insertion keeps automatic fixes to short canonical chains
 292 (`scale-then-PCA`, `neighbors-then-leiden`) and prevents silent reconstruction of multi-stage pipelines.

293 7 §S6 Grader catalog (65 check types)

294 Each task carries a checks list with entries of shape `{"id": ..., "type": <type>, ...kwargs}`. The full `CHECK_DISPATCH` table contains 65 distinct check types, organised in five
 295 families.
 296

297 7.1 §S6.1 Existence checks (10 types)

298 `must_have_obs_keys`, `must_have_var_keys`, `must_have_uns_keys`, `must_have_layers`,
 299 `must_have_layers_regex`, `must_have_obs_keys`, `must_have_obs_keys_regex`,
 300 `must_have_obs_keys_regex`, `must_have_var_keys_regex`, `must_have_uns_keys_regex`,
 301 `must_have_any_container_regex`. They reject pipelines that finish without producing the
 302 expected `AnnData` fields.

303 7.2 §S6.2 Distributional checks (5 types)

304 `x_value_range`, `per_cell_expm1_sum` (rejects an X matrix that is not log-normalised to a tar-
 305 get depth), `layer_dtype_check` (counts must be integer), `var_bool_sum` (HVG count within
 306 tolerance), `var_count_above`.

307 7.3 §S6.3 Biology-anchored checks (12 types)

308 `marker_overlap_in_var` (top-N HVG must include canonical PBMC
 309 / pancreas markers from PanglaoDB), `aucell_celltype_specificity`,
 310 `pseudotime_root_anchored` (the pseudotime field flows from a known progenitor clus-
 311 ter), `obs_count_matching_regex`, `obs_count_matching_regex`, `obs_value_comparison`,
 312 `obs_unique_count`, `obs_unique_subset`, `var_unique_count`, `obs_categorical_match`,
 313 `obs_categorical_overlap_at_least`, `lr_pair_reference`.

314 7.4 §S6.4 Numerical-quality checks (15 types)

315 `clustering_ari` against ground-truth cell-type labels; `clustering_resolution_stability`;
 316 `obs_celltype_silhouette`; `spatial_silhouette`; `pairwise_pseudotime_correlation`;
 317 `obs_shape`; `pca_variance_ratio_top`; `shape_range`; `value_range`;
 318 `obs_dtype_categorical`; `obs_value_topk_overlap`; `cosine_similarity_obs`;
 319 `fraction_obs_value_below`; `fraction_obs_value_above`; `obs_in_categorical`.

320 7.5 §S6.5 Multi-method consensus (8 types)

321 `multi_doublet_consensus`, `batch_correction_silhouette_drop` (scIB-style: silhouette
 322 drop on batch label + cell-type silhouette preservation), `multi_method_directional_agreement`
 323 (velocity), `multi_method_da_agreement` (differential abundance Jaccard),
 324 `multi_method_de_consistency`, `multi_method_annotation_agreement`,
 325 `alpha_diversity_present`, `beta_diversity_present`.

326 7.6 §S6.6 Workflow / trace evidence (15 types)

327 `fine_tune_evidence`, `tool_output_evidence`, `spatial_svg_count`, `var_density`, plus 11
 328 task-specific specialisations.

329 A task's **Pass@1** is the conjunction of all its checks; a task's **fractional score** is the mean. Every check
 330 type has a literature-anchored or numerical justification documented in `tasks_md/<task_id>.md`
 331 — the grader is auditable, not opaque.

332 8 §S7 The system prompt

333 The system prompt the +omicverse arm runs with is 222 lines and embeds three layers of discipline.

334 **1. Discovery discipline.** Call `registry_lookup` and `skill_lookup` at stage boundaries; budget
335 3–4 lookup turns total before coding; treat exact API-name hits as stronger evidence than weak
336 prerequisite-text mentions; if `registry_lookup` returns no match for what looks like a real anal-
337 ysis name, immediately try `skill_lookup` with the same query before assuming the operation is
338 unsupported.

339 **2. Reasoning discipline.** Pre-empt data-shape traps before issuing a command (a "wide" file would
340 dump megabytes per row; an `AnnData` whose orientation is reversed would land masks on the wrong
341 axis); aim for full-pipeline turns rather than many small "inspect, then maybe do" turns; evaluate
342 every tool result against a biological / numerical expectation, not just describe what it is; on a failure,
343 change only the failing line, not the entire script.

344 **3. Coding conventions.** Always start omicverse code with `import omicverse as ov`; prefer
345 registry-surfaced wrappers over manual reinvention; validate prerequisites (`obs`, `obsn`, `uns`, `layers`)
346 before each downstream call; persist final state with `adata.write_h5ad(path)` so downstream
347 evaluation can read it.

348 The non-obvious choices in the prompt are: (i) the explicit budget on lookup turns, which empirically
349 reduces lookup-overuse by an unscaffolded agent that otherwise calls `registry_lookup` for every
350 function; and (ii) the *trust the dispatch wrappers* paragraph, which prevents the agent from re-
351 implementing a method when a wrapper already exists. The full prompt verbatim is in the released
352 `prompts/omicverse_system.md`.

353 9 §S8 Per-model task scope and seeds

354 The canonical v1.1 panel is **uniform** in tasks: 38 tasks per model (A: 5, B: 10, C: 4, E: 6, F: 4, G:
355 5, L: 4) across all seven models. The L-layer foundation-model tasks (added in v1.1 after the v1.0
356 evaluation window) are present on every sweep.

Model	Provider	tasks	seeds
qwen-3B-a3b-256k	Alibaba	38	0
gemini-3.1-flash-lite-preview	Google	38	0–2
gpt-5.5	OpenAI	38	0–2
deepseek-v4-flash	DeepSeek	38	0–2
deepseek-v4-pro	DeepSeek	38	0–2
glm-5.1	Zhipu	38	0–2
MiniMax-M2.7	MiniMax	38	0–2

358 Two ablation arms (`no_registry` = 222-line discipline + matched- length flat docstring dump;
359 `doc_RAG` = all-MiniLM-L6-v2 top-8 retrieval over raw docstrings) were run on `gpt-5.5` and
360 `deepseek-v4-flash` at seed 0; results in §6.5 / Fig. 3.

361 Full task IDs (same set across all sweeps): A01–A05, B01–B10, C01–C04, E01–E06, F01–F04,
362 G01–G05, L01–L04.

363 10 §S9 Sklearn mini-bench: tasks and graders

364 The five sklearn tasks of §7.2, with their full English prompts and grader logic. Two-arm Pass@1
365 results against `deepseek-v4-flash`:

Task	Threshold	Baseline	+ registry
T1 Iris+grid	$\text{acc} \geq 0.90$	PASS 1.000	PASS 1.000
T2 imbalanced	$\text{F1} \geq 0.50$	PASS 0.632	PASS 0.557
T3 regr.+select	$\text{R}^2 \geq 0.55$, +select	FAIL (no model)	PASS 0.996
T4 cluster+sil	$\text{sil} \geq 0.50$	PASS 0.863	PASS 0.845
T5 pipe+CV	$\text{cv-acc} \geq 0.85$, +pipe+scale	FAIL 0.762	PASS 0.926
Pass@1	—	3 / 5 (60%)	5 / 5 (100%)

10.1 §S9.1 T1 — Iris classification with hyperparameter selection

> Train a classifier on $(X_{\text{train}}, y_{\text{train}})$ and evaluate it on $(X_{\text{test}}, y_{\text{test}})$. Use hyperparameter selection (e.g. cross-validated grid / random search) so the chosen model is the best of several candidates. Storage targets: `final_model`, > `final_acc`. Pass: `final_acc` >= 0.90.

Setup: `load_iris()`, 30 % stratified test split, seed 0.

10.2 §S9.2 T2 — Imbalanced binary classification

> Imbalanced $(X_{\text{train}}, y_{\text{train}})$, $(X_{\text{test}}, y_{\text{test}})$ with > positive-class rate $\approx 5\%$. Train a classifier whose F1 > on the positive class is as high as possible. Storage targets: > `final_model`, `final_f1`. Pass: `final_f1` >= 0.50.

Setup: `make_classification(n_samples=2000, weights=[0.95, 0.05], n_informative=8, n_redundant=4)`, seed 0.

10.3 §S9.3 T3 — Regression with feature selection

> $(X_{\text{train}}, y_{\text{train}})$, $(X_{\text{test}}, y_{\text{test}})$; 50 features, ~ 10 > are informative. Build a regressor that performs **feature selection** before fitting. Storage: `final_model`, `final_r2`. > Pass: `final_r2` >= 0.55 AND the model exposes evidence of > feature selection (a `support_ / selected_features_` attribute > or a pipeline step whose name implies selection).

Setup: `make_regression(n_features=50, n_informative=10, noise=8)`, seed 0.

10.4 §S9.4 T4 — Cluster validation by silhouette

> Unlabeled X ($n=600$, $d=8$); the data was generated as a mixture > of well-separated blobs. (i) Reduce to 2 components for > inspection. (ii) Cluster, choosing k by silhouette over a > small range. Storage: `final_labels`, `final_silhouette`. Pass: > `final_silhouette` >= 0.50.

Setup: `make_blobs(centers=4, cluster_std=1.0)`, seed 0.

10.5 §S9.5 T5 — End-to-end Pipeline + stratified k-fold CV

> (X, y) ($n=800$, $d=20$) with mixed-scale features. Build a > Pipeline that scales features and fits a classifier; evaluate > with stratified k -fold CV ($k \geq 5$). Storage: > `final_model` (a Pipeline), `final_cv_acc`. Pass: > `final_cv_acc` >= 0.85 AND the pipeline contains a scaling step.

Setup: `make_classification(n_features=20, n_informative=10, n_redundant=4)` with hand-injected mixed scales ($0.01\text{--}1000\times$) so non-scaled fits underperform.

11 §S10 Why the long tail matters

The framework’s marginal value is largest where the agent’s training-corpus exposure is smallest. Three observations support this framing.

(a) The literature. Jain et al. [16] (arXiv:2407.09726) report that GPT-4o achieves only 38.58% valid invocations on low-frequency APIs versus much higher on common APIs, a roughly 50% relative drop. They show that documentation-augmented generation closes much of the gap when an API is sufficiently indexed by frequency — directly motivating registry-surfaced metadata.

402 **(b) The training-corpus skew is structural.** PyPI download counts (a proxy for code-corpus men-
403 tions) span six orders of magnitude. `sklearn` clocks $\approx 198,570,908$ downloads/month; `omicverse`
404 clocks $\approx 7,620$ /month — a ratio of $\sim 26,000\times$. GitHub stars show a similar gap: `sklearn` has
405 $\sim 65,900$, `omicverse` has ~ 960 . An LLM trained on a code corpus weighted by GitHub / PyPI
406 popularity has seen `sklearn`’s surface tens of thousands of times more often than `omicverse`’s.

407 **(c) The empirical pattern matches.** On `omicverse` the seven-model panel shows a +17.8 pp mean
408 uplift (multi-seed); on `sklearn` the `deepseek-v4-flash` 5-task ablation shows a +40 pp uplift on the
409 two compose-two-primitives tasks but +0 on the three baseline- already-knows tasks (T1 Iris, T2
410 imbalanced, T4 clustering). The +0 cells are saturation — the agent already has the prior. On a niche
411 library every cell is a +0-or-+something contest, and the framework’s value is large.

412 We do not run a third long-tail library experiment in v1 because *it would not change the conclusion* —
413 the `omicverse` evidence is already in the long-tail regime. A second-library pilot (5 tasks on `nibabel`
414 or `biopython`) is a natural v1.1 deliverable.